SAML SSO & Automated User Provisioning in Okta

A hands-on identity federation project — documented by Rithik Kumar

For this practice project I set up real SAML-based Single Sign-On between Okta (as Identity Provider) and a target application, plus automated, API-driven user provisioning on top of it — not just login, but create/update/deactivate lifecycle actions triggered from Okta.

I used Salesforce as the target application for this specific demo, since its free Developer Edition is one of the few platforms that lets you configure a genuinely complete SAML + REST provisioning integration without a paid tier. The configuration pattern here — IdP metadata exchange, Connected App / OAuth provisioning setup, attribute mapping, import reconciliation — is the same pattern used for any SAML 2.0 or SCIM-compatible application (Workday, ServiceNow, Slack Enterprise, Zoom, etc.), so this demonstrates the underlying skill, not a Salesforce-specific one.

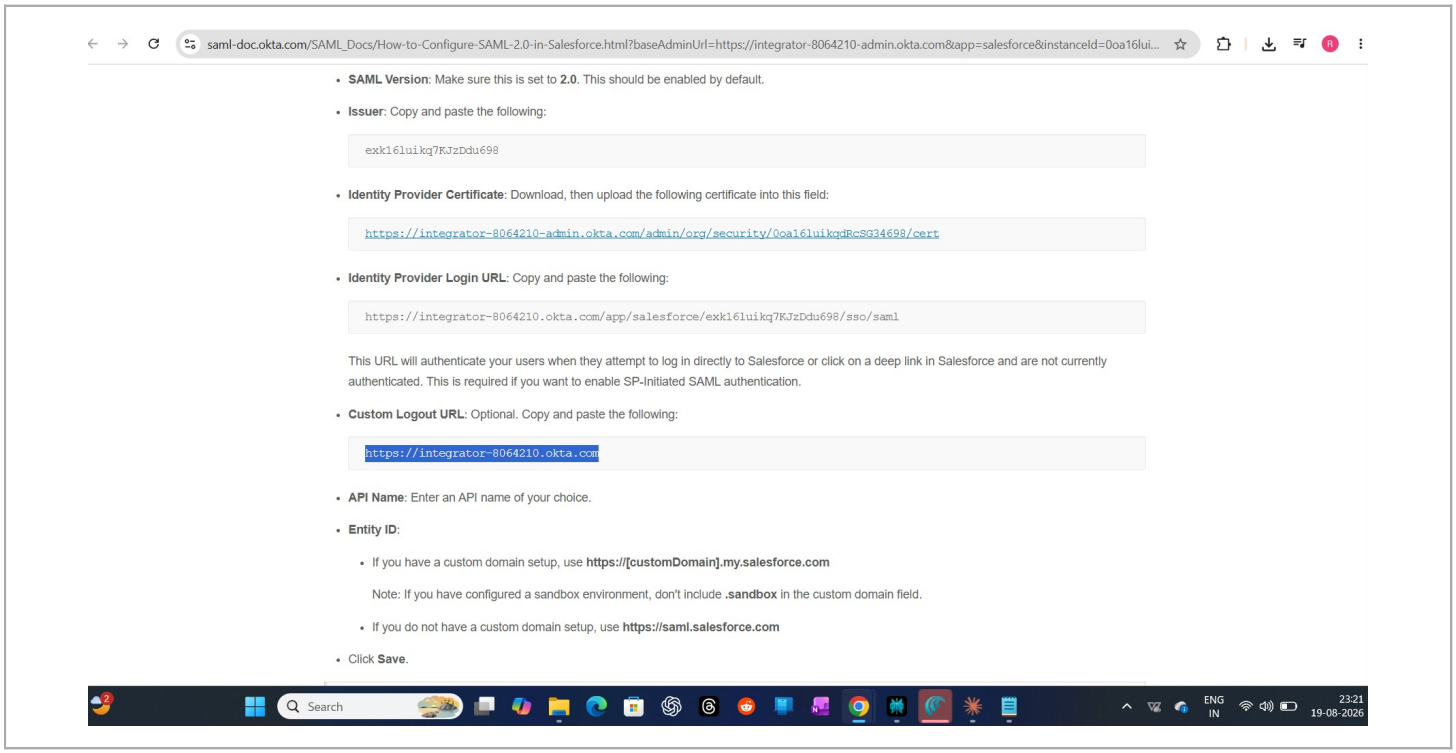
I also independently tested the OAuth 2.0 / OIDC authorization code flow for a separate Okta app using Postman, confirming token issuance end-to-end — kept out of this write-up to avoid repeating the same login concepts twice.

Official references I followed throughout: Okta's SAML configuration guide for Salesforce, and Okta's REST-based provisioning setup guide (both linked in the GitHub repo README).

PHASE 1 — SAML TRUST CONFIGURATION

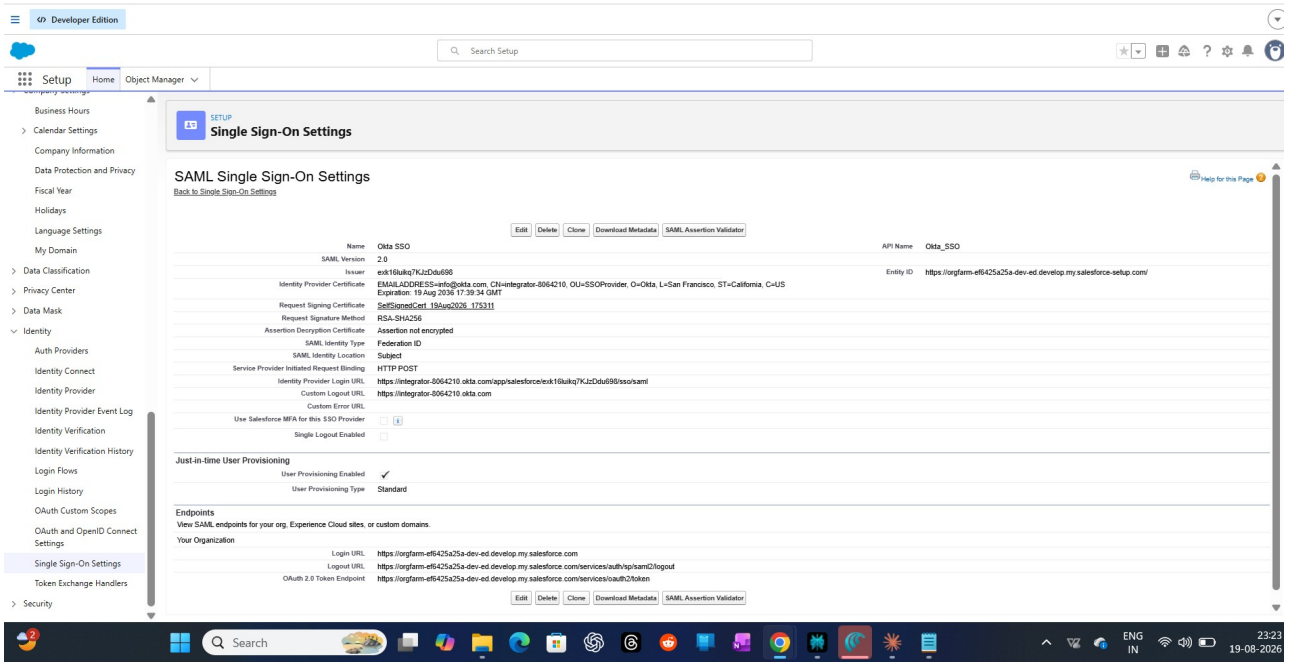
Step 1 — Following Okta's Official SAML Setup Guide

Started with Okta's own SAML configuration docs for the Salesforce integration — it lays out exactly what Salesforce needs: the Issuer, the IdP certificate, and the IdP Login URL. I kept this open in one tab the whole time as a reference.



Step 2 — SAML SSO Settings Saved on the Salesforce Side

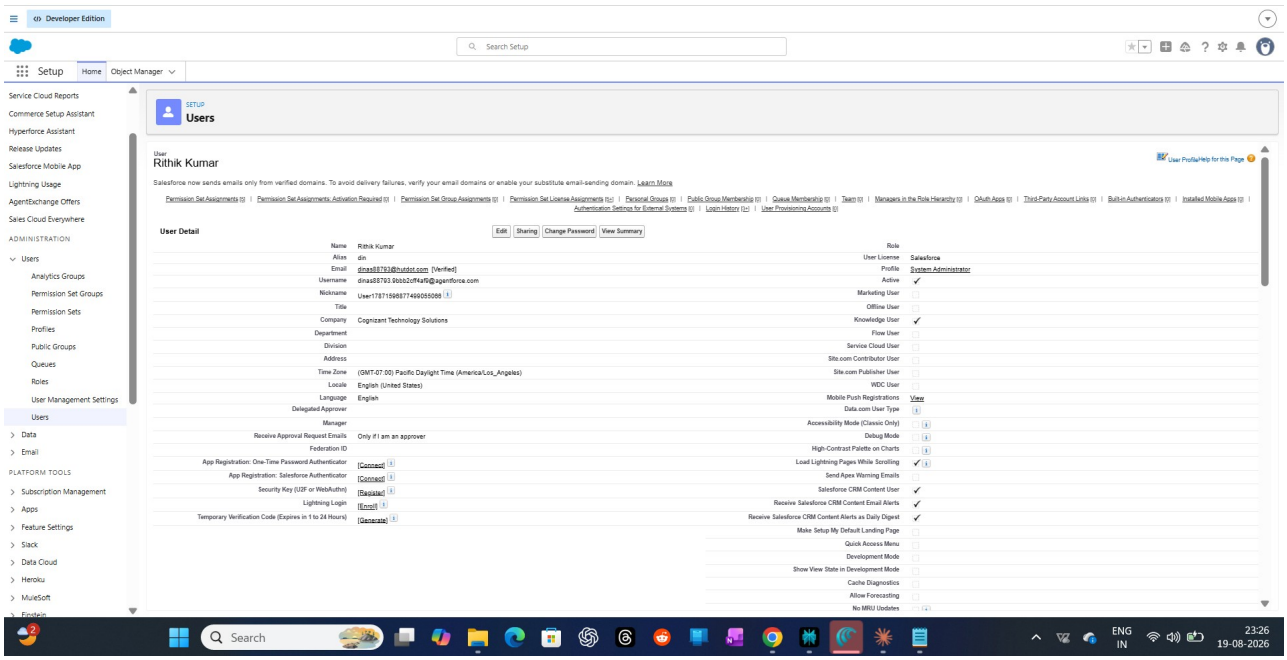
Pasted Okta's Issuer, uploaded the certificate, and set the Identity Provider Login URL inside Salesforce's Single Sign-On Settings. Once saved, Salesforce shows its own Entity ID and the SP endpoints (Login URL, Logout URL, OAuth Token Endpoint) it generated in return — this is the two-way trust handshake between Okta (IdP) and Salesforce (SP) actually completing.



PHASE 2 — SETTING UP AUTOMATED PROVISIONING

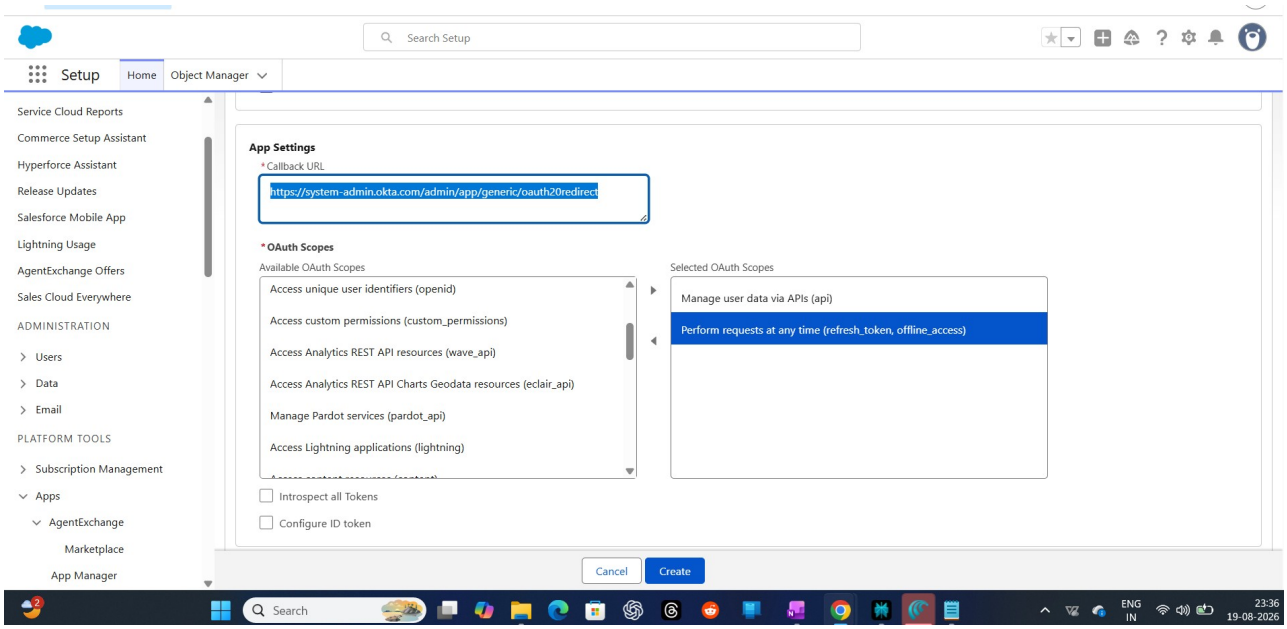
Step 3 — Reviewing the Target Org's User Model First

Before turning on automated provisioning, I checked my own admin user record in Salesforce to understand the fields Salesforce actually tracks (Profile, Role, License type) — this matters because provisioning has to map correctly against these.



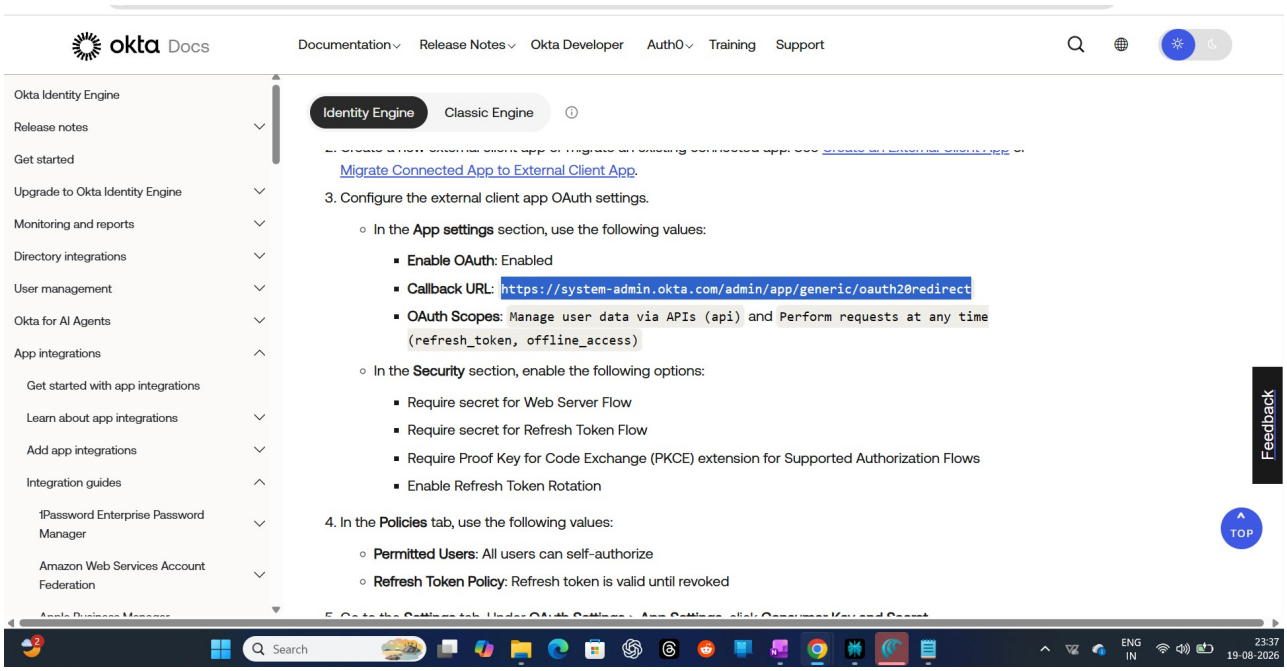
Step 4 — Creating a Connected App for API-Based Provisioning

SAML handles login, but provisioning (auto-creating/updating/deactivating users) needs a separate OAuth-based connection. Created an External Client App in Salesforce with a callback URL and the OAuth scopes Okta needs to manage users via API.



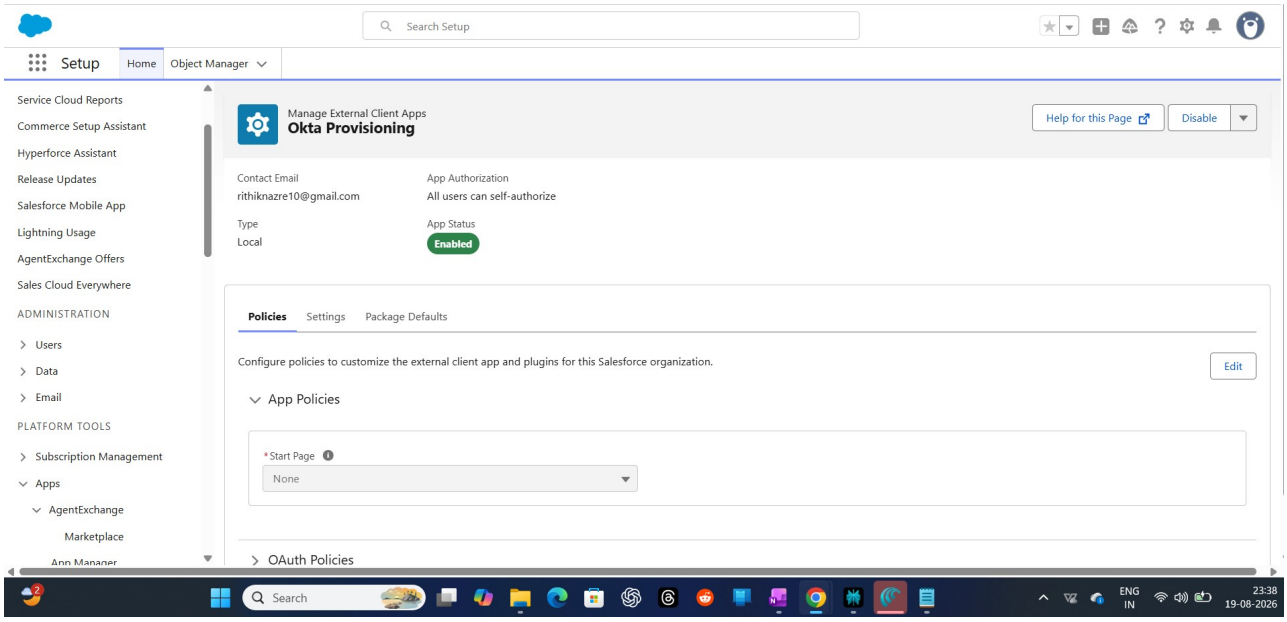
Step 5 — Cross-Checking Against Okta's REST Provisioning Docs

Referenced Okta's help docs for REST-based Salesforce provisioning to confirm the exact callback URL format and scopes expected — didn't want to guess this part.



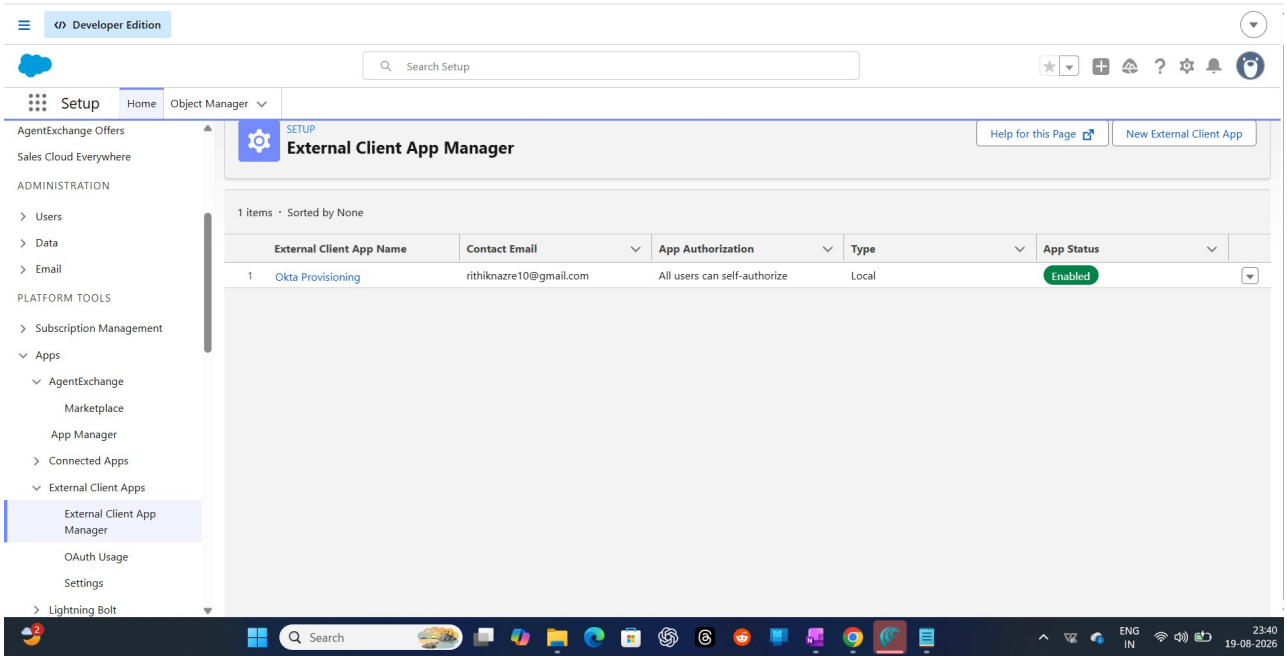
Step 6 — Confirming the Connected App Is Active

Checked the app's policy settings and confirmed its status shows Enabled before trying to connect it from Okta's side.



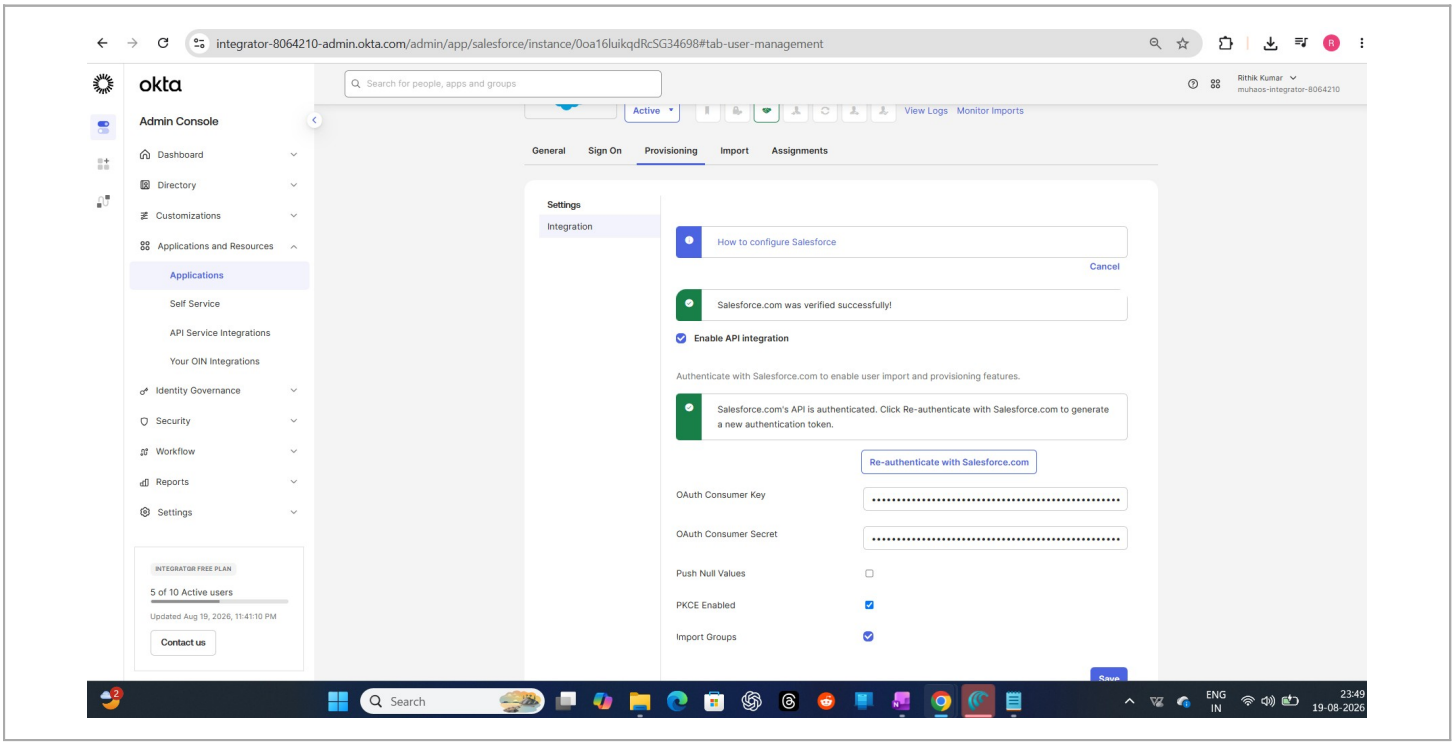
Step 7 — Verifying It Shows Up in Salesforce's App Manager

Double-checked in External Client App Manager that the app is listed and enabled — small sanity check before switching over to Okta.



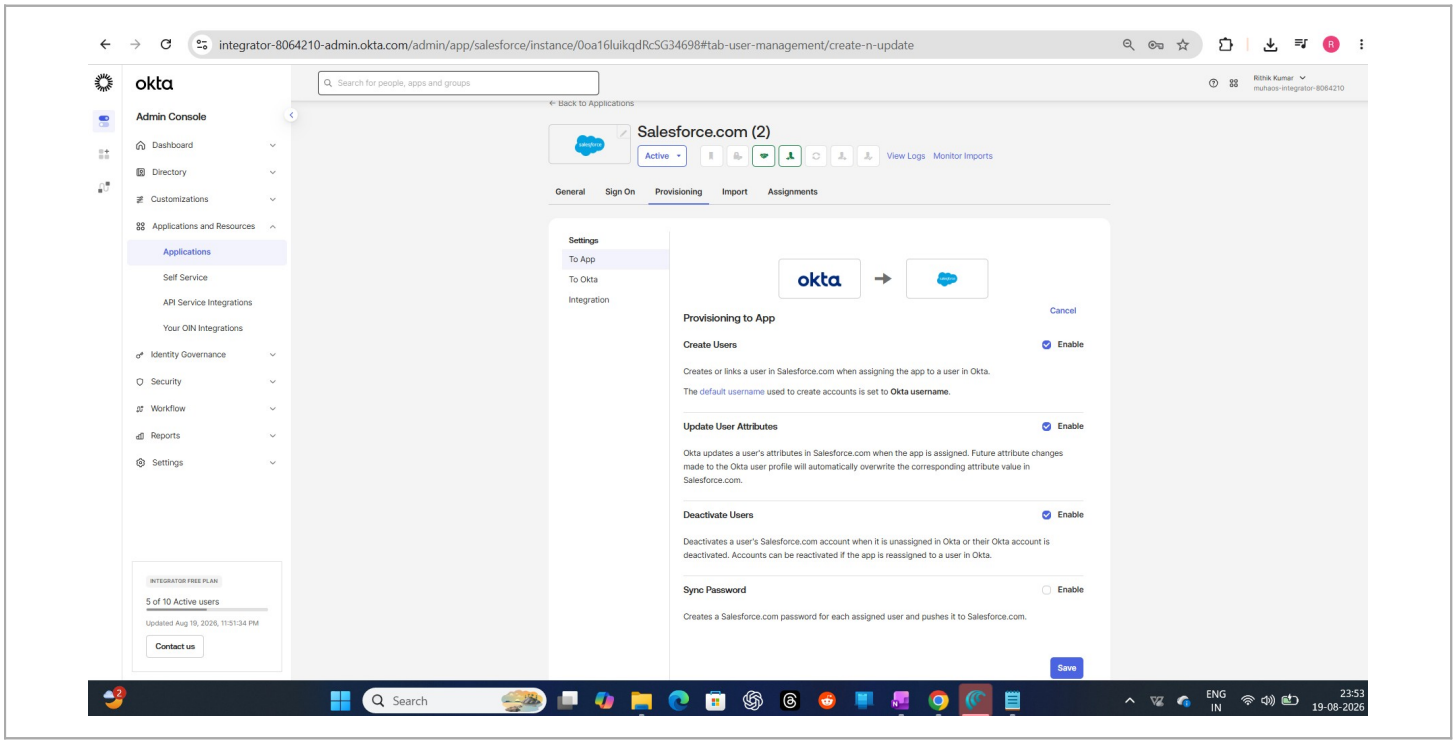
Step 8 — Authenticating Okta Against Salesforce's API

Back in Okta, enabled API integration on the Salesforce app and authenticated using the Connected App's OAuth Consumer Key and Secret. (Not sharing the actual key/secret values here — blurred out in the screenshot, and I'll be deleting this dev app after finishing this practice project anyway.) PKCE and Import Groups are both enabled.



Step 9 — Turning On Create / Update / Deactivate

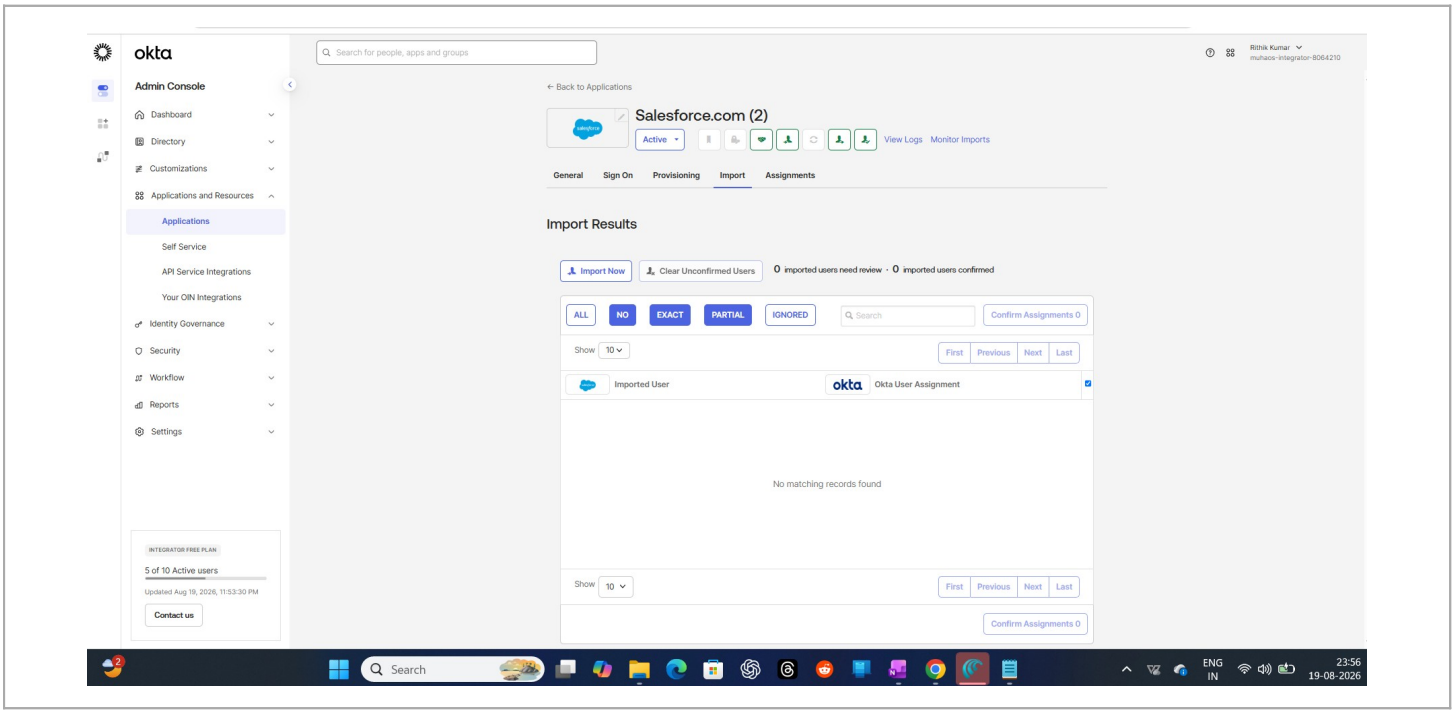
Enabled the three core provisioning actions on the Okta side: Create Users, Update User Attributes, and Deactivate Users — this is what actually makes JML-style automation possible against Salesforce, not just SSO login.



PHASE 3 — IMPORT & RECONCILIATION

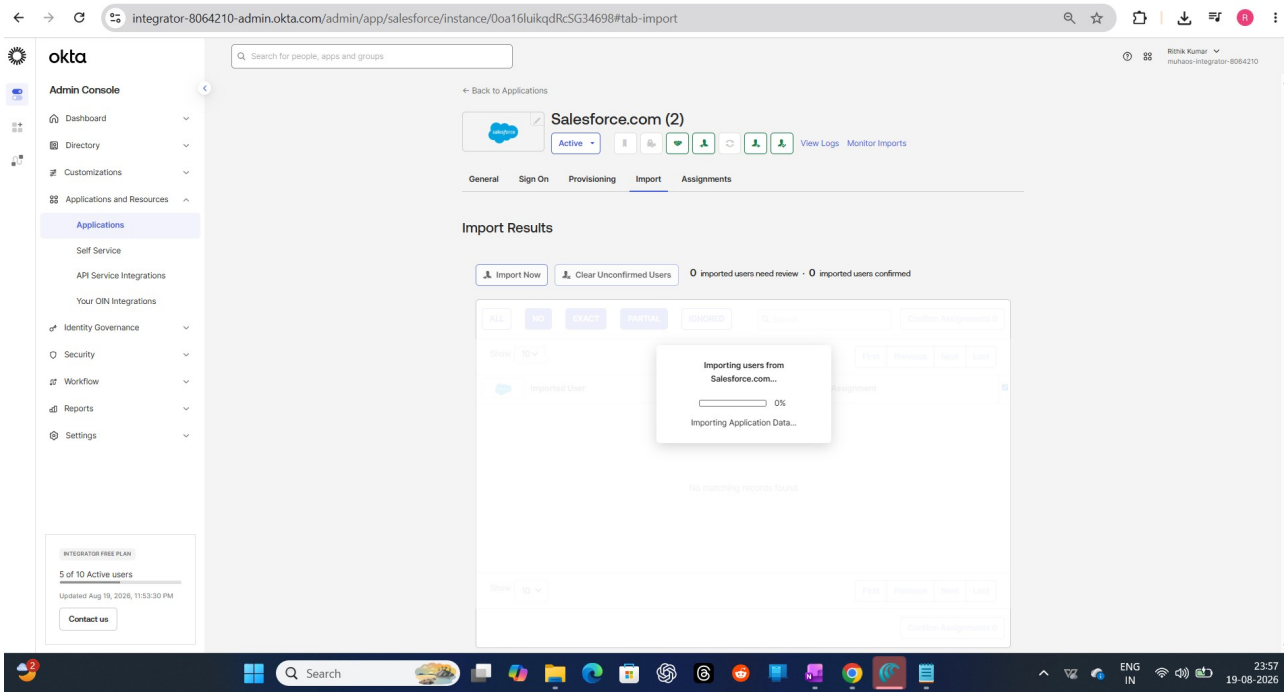
Step 10 — Running the First Import (Nothing Yet)

Ran an initial import from Salesforce into Okta to see what user data would come across. At this point, nothing had synced yet — clean starting point.



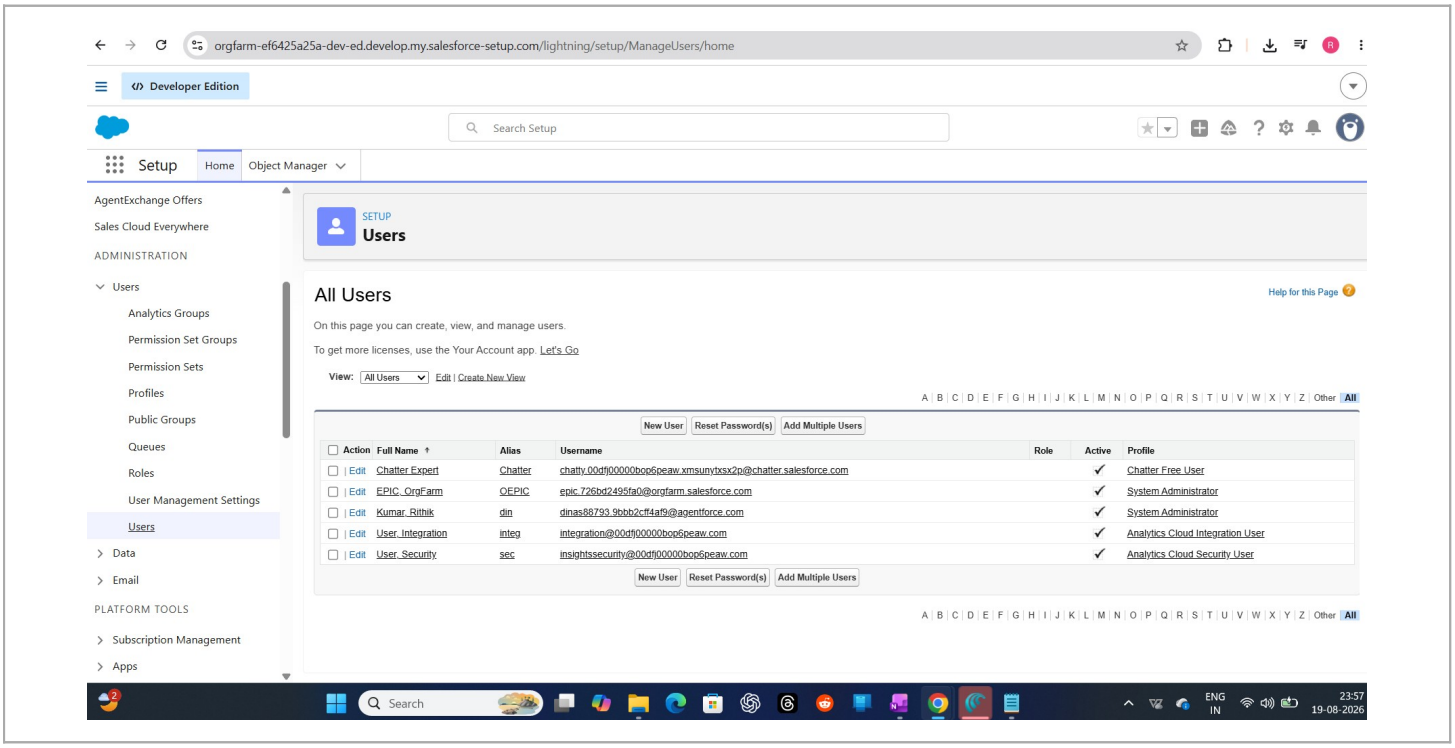
Step 11 — Import In Progress

Kicked off the real import — Okta pulling account data live from Salesforce.



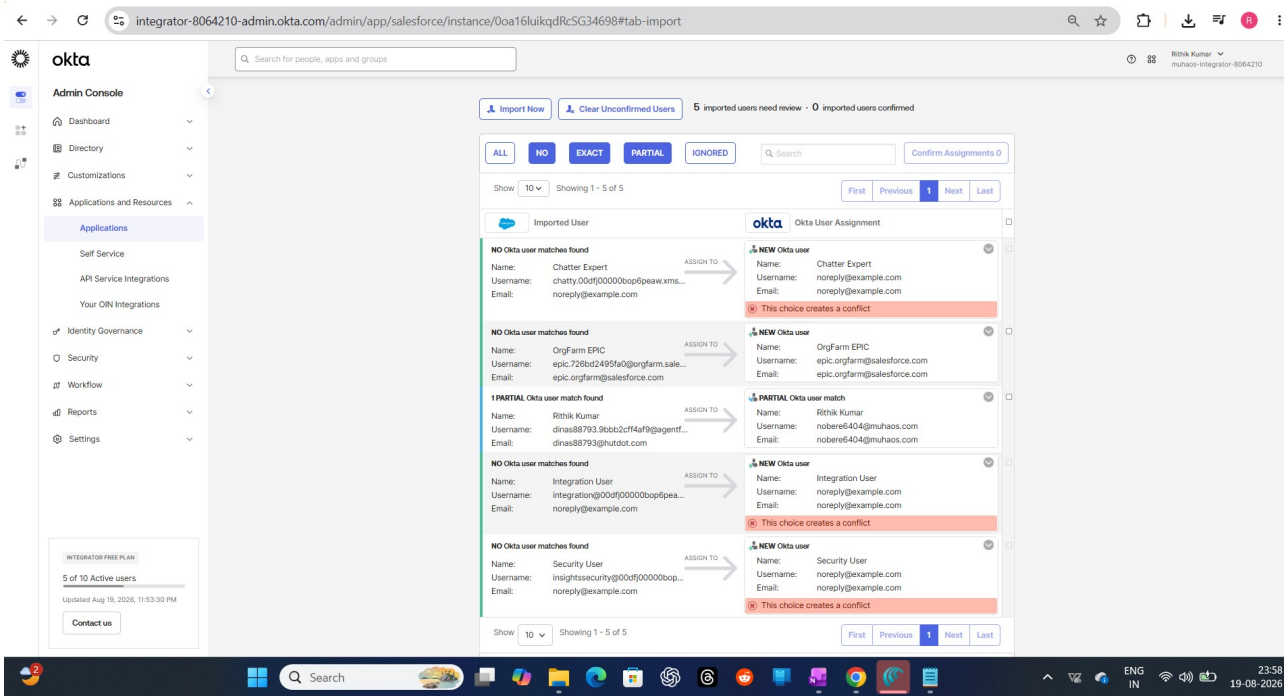
Step 12 — What Salesforce Actually Has on Its Side

Checked Salesforce's own user list directly to see what existed there already (a mix of default system users like Chatter Expert, an integration user, and my own admin account) — useful to know what the import was about to try matching against.



Step 13 — Reviewing the Match Results

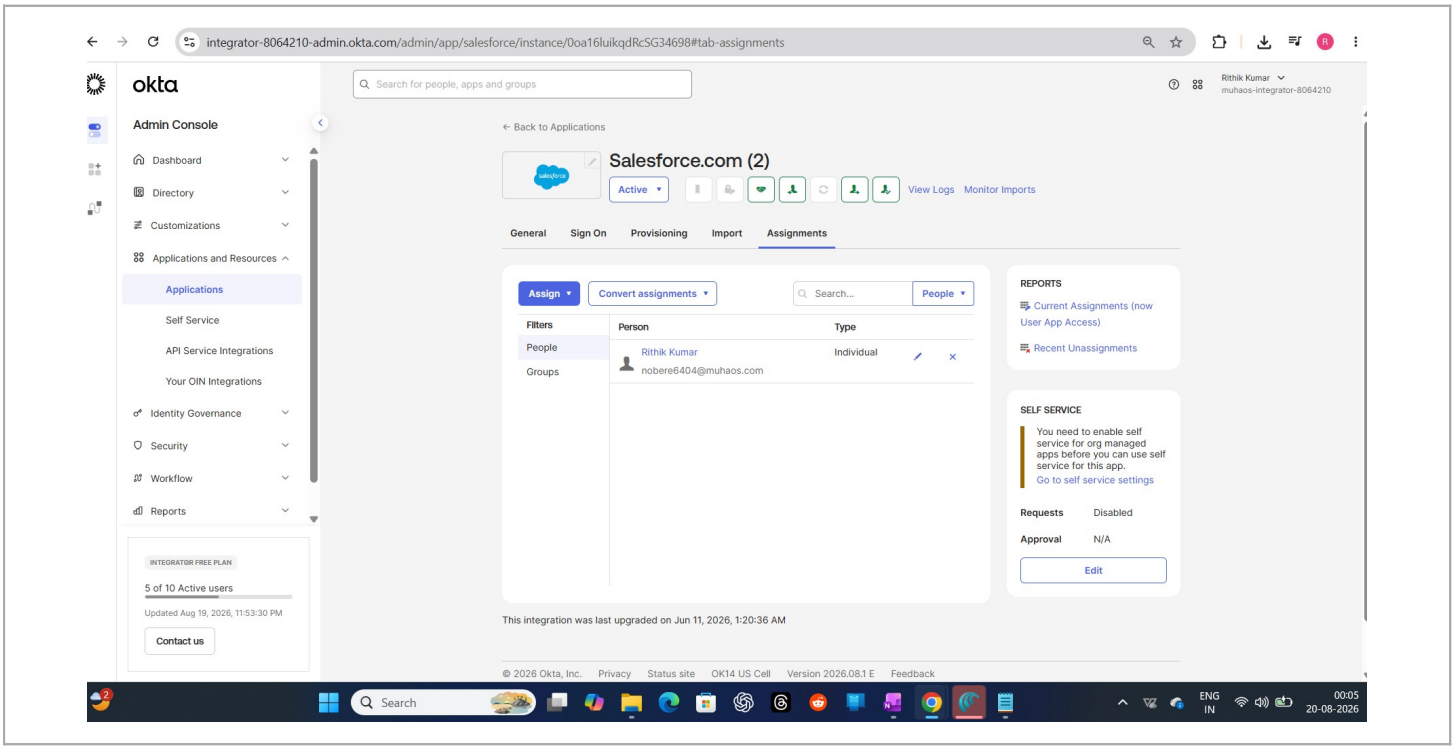
Okta's import screen shows exactly how each Salesforce account was evaluated: most came back as "NO Okta user matches found" (meaning they'd become new Okta identities), but my own account came back as a PARTIAL match — useful for seeing how reconciliation logic actually behaves with real, messy data instead of a clean demo dataset.



PHASE 4 — ASSIGNMENT & PROVISIONING IN ACTION

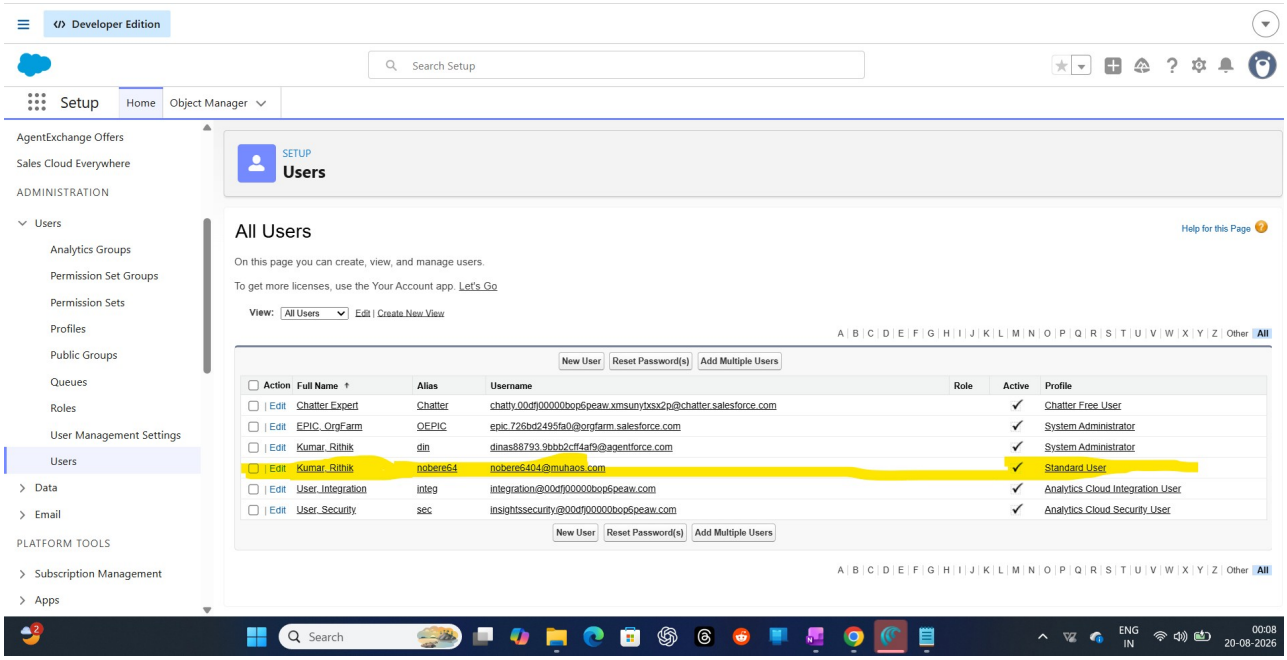
Step 14 — Assigning the App to Trigger Provisioning

Assigned the Salesforce app to my own Okta user. This is the action that actually fires the Create User provisioning call over to Salesforce.



Step 15 — Confirming the User Was Really Created in Salesforce

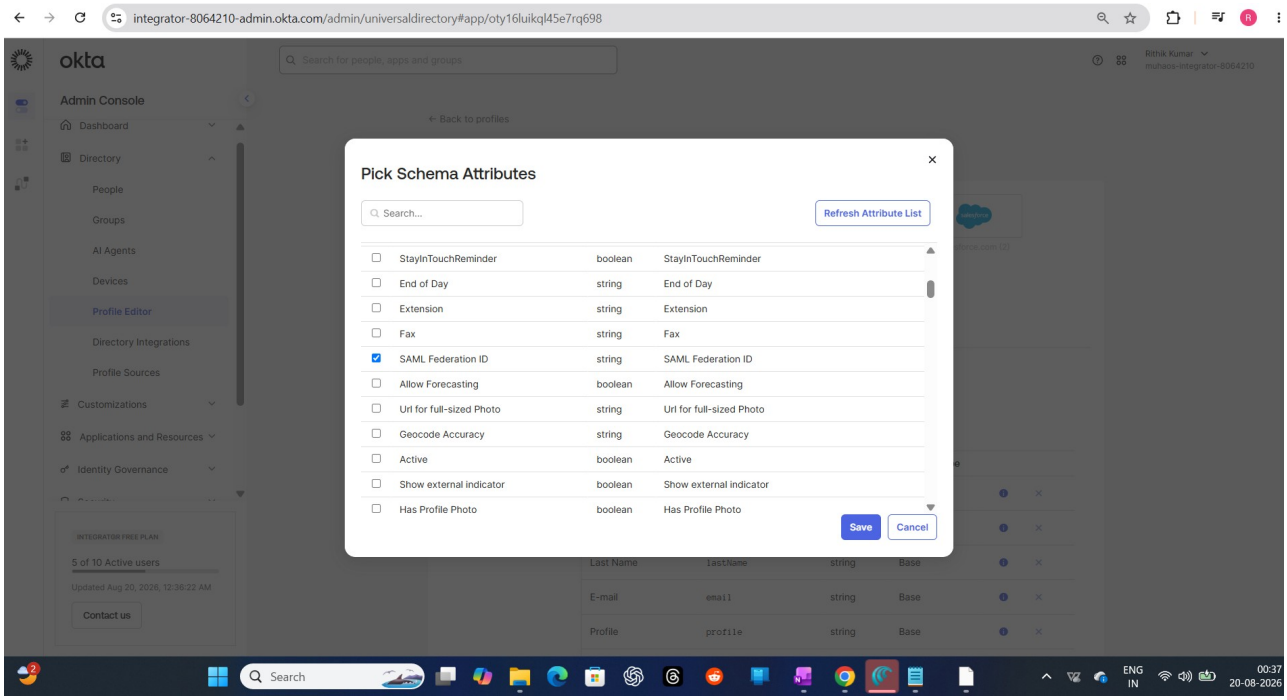
Went back into Salesforce's own user list and found the new record Okta had just provisioned automatically (highlighted) — correct email, correct username, Standard User profile applied. This is the actual proof that the automation works end-to-end, not just configured on paper.



PHASE 5 — ATTRIBUTE MAPPING

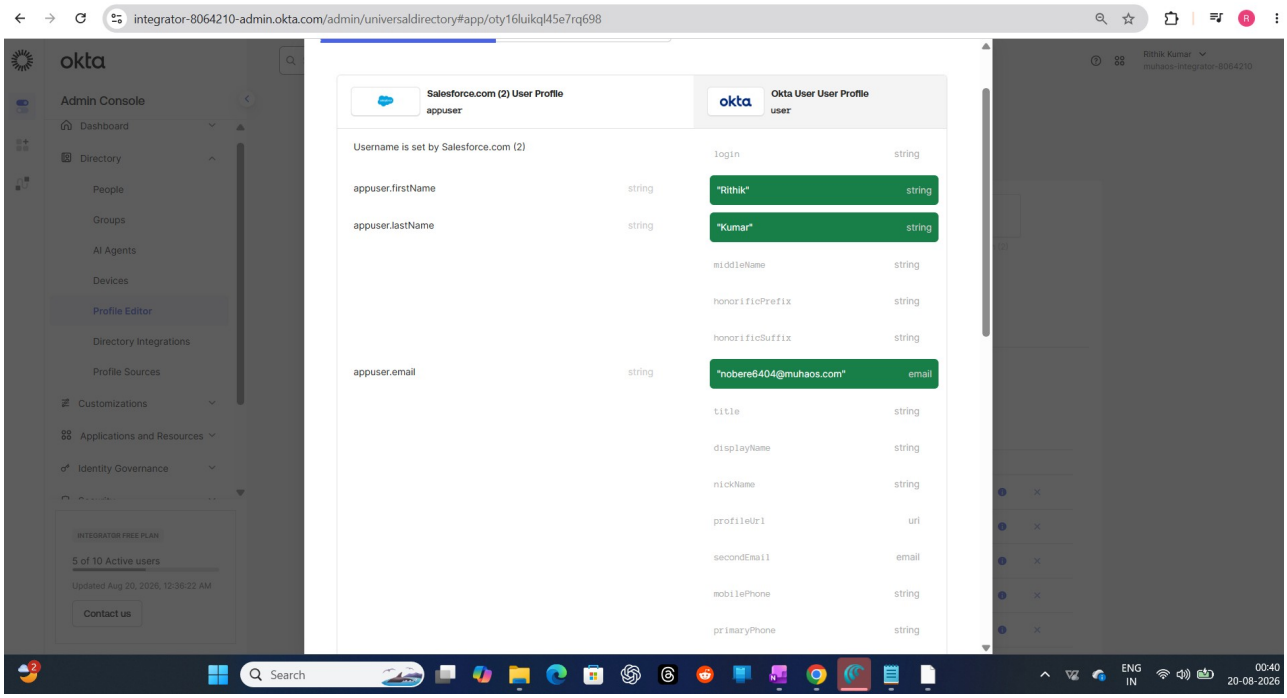
Step 16 — Mapping Profile Attributes Between Systems

Opened Okta's Profile Editor for the Salesforce app and picked which schema attributes should flow between the two systems, including SAML Federation ID — this is what ties the SSO login identity to the correct provisioned account.



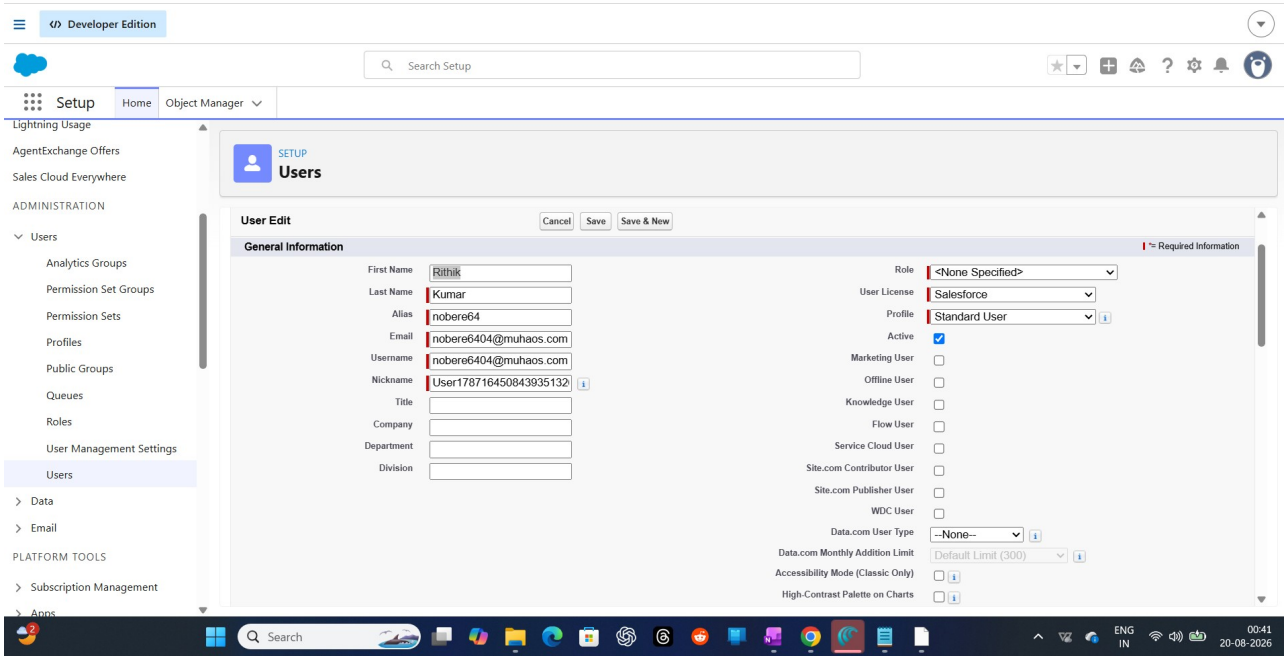
Step 17 — Verifying firstName / lastName / email Actually Map Correctly

Checked the live mapping between Salesforce's appuser fields and Okta's user profile — confirmed my real first name, last name, and email were correctly flowing across, not placeholder values.



Step 18 — Final Provisioned User Record in Salesforce

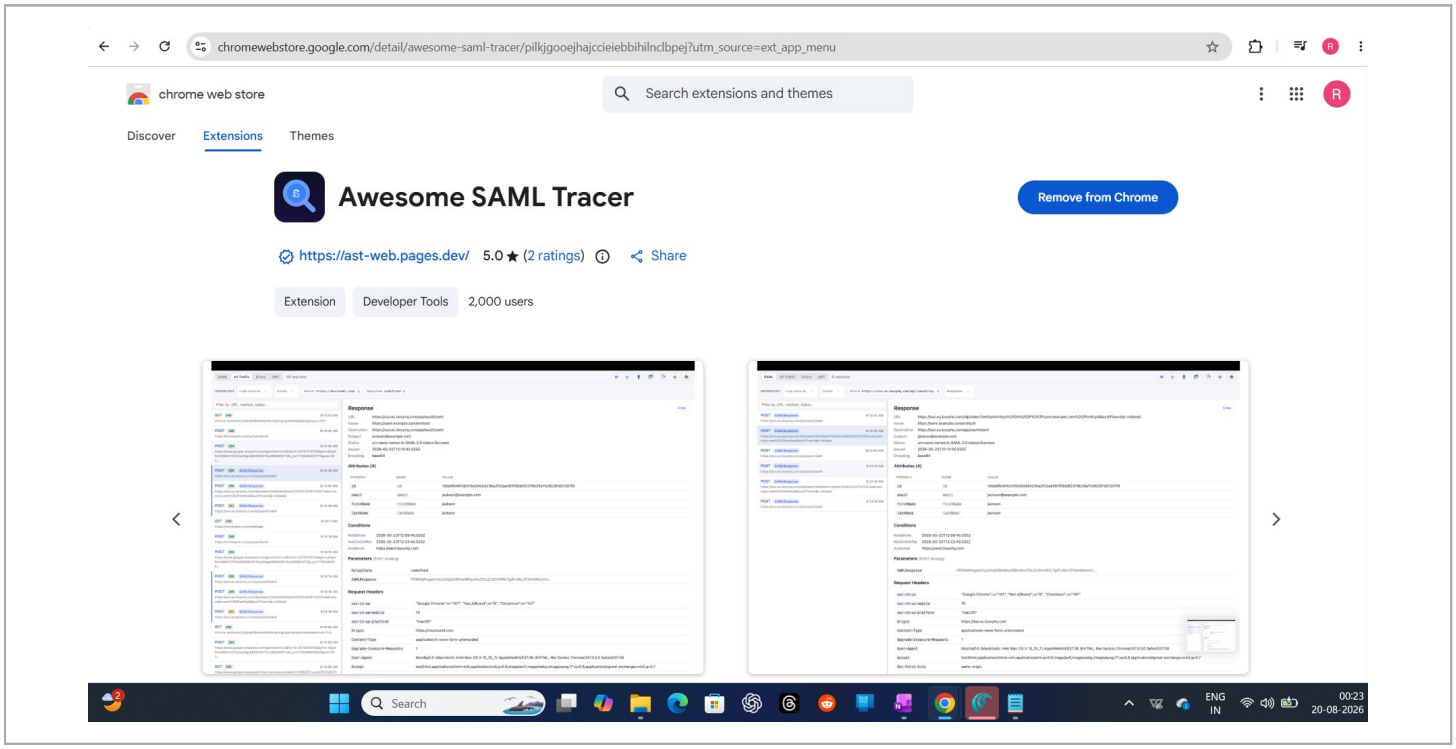
Opened the provisioned user's record directly in Salesforce to confirm everything landed correctly — profile, license type, active status, all set as expected by the automation.



PHASE 6 — INDEPENDENT VALIDATION

Step 19 — Installing a SAML Tracer for Third-Party Verification

Rather than just trusting that the integration "looked right" in the admin consoles, I installed the Awesome SAML Tracer Chrome extension to independently inspect the actual SAML assertions being exchanged during login — then cross-checked that output against Salesforce's own SAML Assertion Validator tool. This lets you pinpoint exactly which part of the exchange is failing if something ever breaks, instead of re-checking the whole integration blind.



What This Project Actually Demonstrates

This wasn't just flipping a toggle labeled "Enable SSO." It covers the full real-world sequence: establishing IdP/SP trust via SAML metadata exchange, setting up a separate OAuth-based Connected App for provisioning, running an import to reconcile existing target-system accounts against Okta identities (including handling partial matches), triggering live provisioning, verifying attribute mapping end-to-end, and independently validating the SAML assertion with a third-party tool instead of just trusting the admin UI.

The reconciliation step in Phase 3 was the most useful part to actually do hands-on — real orgs never have a clean slate, and seeing how Okta classifies existing accounts as new / partial / exact matches is exactly the kind of judgment call that comes up in real migrations.
